Ingegneria del Software — Soluzione del Tema 15/01/2024

Esercizio 1

Si consideri la seguente classe `Shop` che gestisce l'acquisto di prodotti (`Product`) online. Un utente (`User`) può aggiungere prodotti al proprio carrello con il metodo `addToCart` e infine confermare l'acquisto con il metodo `checkout`.

```
public class Shop {
  // Ritorna il numero di pezzi disponibili nello Shop per quel prodotto.
  public /*@ pure @*/ int getAvailability(Product p) { .. }

  // Ritorna la lista di prodotti presenti nel carrello per l'utente u (una lista vuota se l'utente u
  // non ha un suo carrello).
  // In caso l'utente abbia aggiunto un prodotto al carrello n volte, questo apparira' n volte nel risultato.
  public /*@ pure @*/ List<Product> getCart(User u) { .. }

  // Aggiunge il prodotto p nel carrello dell'utente u (in ultima posizione), riducendo di uno la
  // disponibilita' residua del prodotto nello Shop.
  // Lancia una NotAvailableException se il prodotto non e' disponibile (ovvero la sua disponibilita' e' 0)
  public void addToCart(User u, Product p) throws NotAvailableException { .. }

  // Se la carta di credito ha sufficiente credito residuo procede con l'acquisto,
  // svuotando il carrello per l'utente u e riducendo di conseguenza il credito residuo della carta.
  // Lancia una InsufficientCreditException se la carta di credito non ha sufficiente credito residuo,
  // svuotando il carrello per l'utente u e rendendo nuovamente disponibili i prodotti in esso contenuti.
  public void checkout(User u, CreditCard c) throws InsufficientCreditException { .. }
}

public /*@ pure @*/ class Product {
  // Ritorna il prezzo del prodotto
  public int getPrice() { .. }
}

public class CreditCard {
  // Ritorna il credito residuo sulla carta di credito
  public /*@ pure @*/ int getResidualCredit() { .. }
}
```

Domanda a)

Si specifiche in JML il metodo `addToCart`.

Soluzione

```
//@ requires u != null && p != null
//@
//@ ensures \old(getAvailability(p)) >= 1 && getCart(u).size() == \old(getCart(u)).size() + 1 &&
//@ getCart(u).get(getCart(u).size() - 1).equals(p) &&
//@ (\forall int i; i >= 0 && i<\old(getCart(u).size());
//@    getCart(u).get(i).equals(\old(getCart(u)).get(i)) ) &&
//@ (\forall Product p1; p1 != null;
//@    p1.equals(p) ==> getAvailability(p1) == \old(getAvailability(p1)) - 1 &&
//@    !p1.equals(p) ==> getAvailability(p1) == \old(getAvailability(p1)) )
//@ (\forall User u1; u1 != null && !u1.equals(u);
//@    getCart(u1).size() == \old(getCart(u1).size()) &&
//@    (\forall int i; i>=0 && i<getCart(u1).size(); getCart(u1).get(i).equals(\old(getCart(u1).get(i)))))
//@
//@ signals (NotAvailableException e) \old(getAvailability(p)) < 1 &&
//@ (\forall Product p1; p1 != null;
//@    getAvailability(p1) == \old(getAvailability(p1)) ) &&
//@ (\forall User u; u != null;
//@    getCart(u).size() == \old(getCart(u)).size() &&
//@    (\forall int i; i>=0 && i<getCart(u).size(); getCart(u).get(i).equals(\old(getCart(u).get(i)))))
public void addToCart(User u, Product p) throws NotAvailableException { .. }
```

Domanda b)

Si specifiche in JML il metodo `checkout`.

Soluzione

```
//@ requires u != null && c != null
//@
//@ ensures \old(c.getResidualCredit()) >=
```

```
//@   (\sum int i; i>=0 && i<\old(getCart(u).size()); \old(getCart(u).get(i).getPrice())) &&
//@ c.getResidualCredit()==\old(c.getResidualCredit()) -
//@   (\sum int i; i>=0 && i<\old(getCart(u).size()); \old(getCart(u).get(i).getPrice())) &&
//@ (\forall Product p; p != null; getAvailability(p) == \old(getAvailability(p))) &&
//@ (\forall User u1; u1 != null;
//@    u1.equals(u) ==> getCart(u1).isEmpty() &&
//@    !u1.equals(u) ==> getCart(u1).size() == \old(getCart(u1)).size() &&
//@                      (\forall int i; i>=0 && i<getCart(u1).size();
//@                            getCart(u1).get(i).equals(\old(getCart(u1).get(i)))))
//@
//@ signals (InsufficientCreditException e) \old(c.getResidualCredit()) <
//@   (\sum int i; i>=0 && i<\old(getCart(u).size()); \old(getCart(u).get(i).getPrice())) &&
//@ c.getResidualCredit()==\old(c.getResidualCredit()) &&
//@ (\forall User u1; u1 != null;
//@    u1.equals(u) ==> getCart(u1).isEmpty() &&
//@    !u1.equals(u) ==> getCart(u1).size() == \old(getCart(u1)).size() &&
//@                      (\forall int i; i>=0 && i<getCart(u1).size();
//@                            getCart(u1).get(i).equals(\old(getCart(u1).get(i)))))
//@ (\forall Product p; p != null; getAvailability(p) == \old(getAvailability(p)) +
//@                     (\num_of int i; i>=0 && i<\old(getCart(u)).size(); \old(getCart(u).get(i)).equals(p)))
public void checkout(User u, CreditCard c) throws InsufficientCreditException { .. }
```

Domanda c)

Si consideri la seguente implementazione della classe `Shop`, che fa uso di una mappa `prodAvailability` per memorizzare la disponibilità dei prodotti e di una mappa `carts` per memorizzare i carrelli degli utenti. Per tale implementazione, si specifichino l'invariante di rappresentazione e la funzione di astrazione.

```
public class Shop {
  Map<Product, Integer> prodAvailability;
  Map<User, List<Product>> carts;
... }
```

Soluzione

L'invariante di rappresentazione specifica che le liste non sono nulle e non contengono chiavi o valori nulli. La disponibilità dei prodotti deve essere positiva e il carrello non deve contenere una lista vuota o una lista con elementi nulli.

```
//@ private invariant
//@ prodAvailability != null && !prodAvailability.containsKey(null) &&
//@ (\forall Product p; prodAvailability.containsKey(p);
//@    prodAvailability.get(p) != null && prodAvailability.get(p) > 0 ) &&
//@ carts != null && !carts.containsKey(null) &&
//@ (\forall User u; carts.containsKey(u);
//@    carts.get(u) != null && !carts.get(u).isEmpty() && !carts.get(u).contains(null) )
```

La funzione di astrazione definisce il valore dei metodi base (`getAvailability` e `getCart`) sulla base degli attributi privati. Tutti gli altri metodi sono definiti a partire da questi.

```
//@ private invariant
//@ (\forall Product p; p != null; getAvailability(p) == (prodAvailability.containsKey(p) ?
//@    prodAvailability.get(p) : 0 )) &&
//@ (\forall User u; u != null;
//@    carts.containsKey(u) ==> (getCart(u).size() == carts.get(u).size() &&
//@                              (\forall int i; i>=0 && i<carts.size();
//@                                 getCart(u).get(i) == carts.get(u).get(i) ) &&
//@    !carts.containsKey(u) ==> getCart(u).isEmpty() ) )
```

Domanda d)

Si consideri una classe `RemovableCartShop`, che aggiunge un metodo `void removeFromCart(User u, Product p)` a `Shop`. Tale metodo rimuove tutte le occorrenze di `p` dal carrello dell'utente `u`, ripristinandone la disponibilità. È possibile definire `RemovableCartShop` come sottoclasse di `Shop` in accordo con il principio di sostituzione di Liskov? Motivare la propria risposta.

Soluzione

La classe `RemovableCartShop` può essere definita come sottoclasse di `Shop`. Essa è un'estensione pura e non viola alcuna proprietà invariante o evolutiva di `Shop`. Anche in `Shop` è infatti possibile rimuovere prodotti dal carrello rendendoli di nuovo disponibili attraverso l'invocazione del metodo `checkout` con una carta di credito con credito insufficiente. Tale invocazione, seguita da una opportuna sequenza di invocazioni che rimetta nel carrello tutti i prodotti prima presenti meno il prodotto `p`, è equivalente alla chiamata del nuovo metodo (che quindi non cambia il comportamento atteso).

Esercizio 2

Si consideri la seguente classe `Students` usata per registrare i voti di un gruppo di studenti.

```
public class Students {
  private List<String> studNames;
  private List<Integer> scores;
  public Students() {
    studNames = new ArrayList<>();
    scores = new ArrayList<>();
  }
  public int getScore(String studName) {
    synchronized(studNames) {
      int pos = studNames.indexOf(studName);
      if(pos==-1) return -1;
      else return scores.get(pos);
    }
  }
  public void addScore(String studName, int score) throws IllegalArgumentException {
    if(getScore(studName) != -1) throw new IllegalArgumentException();
    synchronized(studNames) {
      studNames.add(studName);
      scores.add(score);
    }
  }
  public void clear() {
    synchronized(studNames) {
      studNames.clear();
      scores.clear();
    }
  }
}
```

Domanda a)

La sincronizzazione proposta evita il verificarsi di accessi concorrenti a zone di memoria condivise da parte di diversi thread? Motivare la propria risposta ed in caso di risposta negativa suggerire come sia possibile correggere il codice proposto, modificando al più un metodo.

Soluzione

La classe è sincronizzata in maniera da impedire accessi concorrenti ad aree di memoria condivisa.

Domanda b)

La sincronizzazione proposta garantisce che non possano essere inseriti duplicati (studenti con il medesimo nome)? Motivare la propria risposta ed in caso di risposta negativa suggerire come sia possibile correggere il codice proposto, modificando al più un metodo.

Soluzione

Il metodo `addScore` non è atomico e non garantisce quindi l'assenza di duplicati. È infatti possibile che due thread chiamino contemporaneamente il metodo cercando di aggiungere uno studente con il medesimo nome. Entrambi i thread potrebbero passare il test presente nell'istruzione `if` a inizio metodo, rilevando che non è presente uno studente con quel nome, per poi (in sequenza) acquisire il lock su `studScores` e aggiungere lo studente e il relativo voto alla struttura dati, dando così luogo ad una duplicazione. La cosa può facilmente essere corretta modificando il metodo come segue:

```
public void addScore(String studName, int score) throws IllegalArgumentException {
  synchronized(studNames) {
    if(getScore(studName) != -1) throw new IllegalArgumentException();
    studNames.add(studName);
    scores.add(score);
  }
}
```

Domanda c)

Si modifichi il metodo `addScore` perchè sospenda il chiamante invece di sollevare l'eccezione. Indicare se altri metodi devono essere modificati e come.

Soluzione

```
public void addScore(String studName, int score) throws InterruptedException {
  synchronized(studNames) {
    while(getScore(studName) != -1) studNames.wait();
    studNames.add(studName);
    scores.add(score);
  }
}
```

Occorre anche inserire la chiamata `studNames.notifyAll()` nel metodo `clear`.

Esercizio 3

Si considerino le seguenti classi Java, di cui omettiamo l'ovvia implementazione:

```
public class Book {
  public float price() { .. }
  public String title() { .. }
  public String author() { .. }
  public int year() { .. }
}

public class  Library {
  public Set<Book> getAllBooks() { .. }
}
```

Rispondere alle domande seguenti usando **esclusivamente** i costrutti della *programmazione funzionale* (*senza quindi usare while, if.. else, for,* ecc.)

Domanda a)

Implementare il metodo:

```
public static String mostExpensive(Library library) throws IllegalArgumentException
```

che restituisce l'autore del libro più costoso della biblioteca `library`. Se la biblioteca è vuota, il metodo lancia l'eccezione `IllegalArgumentException`.

Soluzione

```
public static String mostExpensive(Library library) throws IllegalArgumentException {
  return library.getAllBooks()
    .stream()
    .max((b1, b2) -> Float.compare(b1.price(), b2.price()))
    .map(Book::author)
    .orElseThrow(() -> new IllegalArgumentException("Library is empty"));
```

Domanda b)

Implementare il metodo:

```
public static Set<String> titlesByAuthor(Library library, String author, int year)
```

che restituisce l'insieme dei titoli dei libri (nella biblioteca `library`) scritti dall'autore `author` prima dell'anno `year`.

Soluzione

```
public static Set<String> titlesByAuthor(Library library, String author, int year) {
  return library.getAllBooks()
    .stream()
    .filter(book -> book.author().equals(author) && book.year() < year)
    .map(Book::title)
    .collect(Collectors.toSet());
}
```

Esercizio 4

Si consideri il seguente metodo:

```
1  public static boolean doSomething(int num, int i) {
2    boolean flag = false;
3    if (num<=2 || i<2)
4      return false;
5    while (i<=num/2 && !flag) {
6      if (num%i == 0) {
7        flag = true;
8      }
9      ++i;
10   }
11   return flag;
12 }
```

Domanda a)

Si determini un insieme *minimo* di test per garantire la copertura delle istruzioni (statement coverage).

Soluzione

Servono due casi di test: un caso per uscire all'istruzione 4, ad esempio (0, 2), e un caso per entrare nel while ed eseguire l'istruzione 7, ad esempio (4, 2).

Domanda b)

Si determini un insieme *minimo* di test per garantire la copertura delle decisioni (edge coverage).

Soluzione

I due casi precedenti non coprono la decisione false dell'if alla riga 6. Si può modificare il caso (4, 2) in (9, 2). Alla prima iterazione del ciclo la decisione alla riga 6 è false, alla seconda iterazione è true (con uscita dal ciclo).

Domanda c)

Si determini un insieme *minimo* di test per garantire la copertura delle decisioni e delle condizioni (edge and condition coverage).

Soluzione

Per coprire l'unica condizione non ancora testata del primo if (`(num>2 && i<2)`), occorre aggiungere un nuovo caso ai due precedenti, ad esempio (3, 0).

Le condizioni del while testate dal secondo caso del punto (b): (9, 2) sono `(i<=num/2 && !flag)` alla prima iterazione e `(i<=num/2 && flag)` all'uscita dal ciclo; per testare il caso rimanente in cui si esce dal ciclo con `i>num/2` basta per esempio il caso (3, 2).

In definitiva, servono almeno 4 casi di test: i due presenti al punto (b), ovvero (0, 2) e (9, 2), a cui si aggiungono (3, 0) e (3, 2).